

CHAPTER 03

OpenCV 주요 클래스

OpenCV 4로 배우는 컴퓨터 비전과 머신 러닝

3장 OpenCV 주요 클래스

3.1 기본 자료형 클래스

3.2 Mat 클래스

3.3 Vec과 Scalar 클래스

3.4 InputArray와 OutputArray 클래스

3.2 Mat 클래스

행렬의 원소 값 참조

- OpenCV 라이브러리는 다양한 영상 처리에 필요한 기능을 구현한 클래스와 함수를 제공함
- OpenCV에서 제공하는 기능 외에 사용자가 직접 자신만의 알고리즘을 구현하여 적용해야 하는 경우도 자주 발생함
- 이를 위하여 영상의 픽셀 값을 참조하는 기능이 필요함
- OpenCV는 Mat 객체에 저장된 행렬 원소 값을 참조하고 값을 변경할 수 있는 다양한 인터페이스를 제공함

0	2	7	5
4	-3	8	6
-1	10	-5	-20

3x4 행렬

화면 출력
값 입력

행렬의 원소 값 참조 : Mat::at() 함수이용

```
template<typename _Tp> _Tp& Mat::at(int y, int x)
```

- y 참조할 행 번호
- x 참조할 열 번호
- 반환값 (_Tp& 타입으로 형 변환된) y번째 행, x번째 열의 원소 값(참조)

- Mat::at() 함수는 템플릿 함수이기 때문에 호출할 때에는 행렬 원소의 자료형(_Tp)을 명시적으로 지정해야 함
- 매개변수 y의 범위는 0부터 (행갯수-1), x의 범위는 0부터 (열갯수-1)
- 행렬에서 y행 x열의 원소 -> 2차원 영상좌표계에서 좌표 (x, y)
- 리턴형이 참조형이므로 리턴값은 원소값이 저장된 공간을 의미함

행렬의 원소 값 참조 : Mat::at() 함수이용

- Mat 객체에 저장된 행렬의 type이 `CV_8UC1`인 경우
 - mat1 객체에서 첫번째 원소는 `mat1.at<uchar>(0, 0)` 형태로 표현함

```
Mat mat1(3, 4, CV_8UC1, Scalar(0));
```

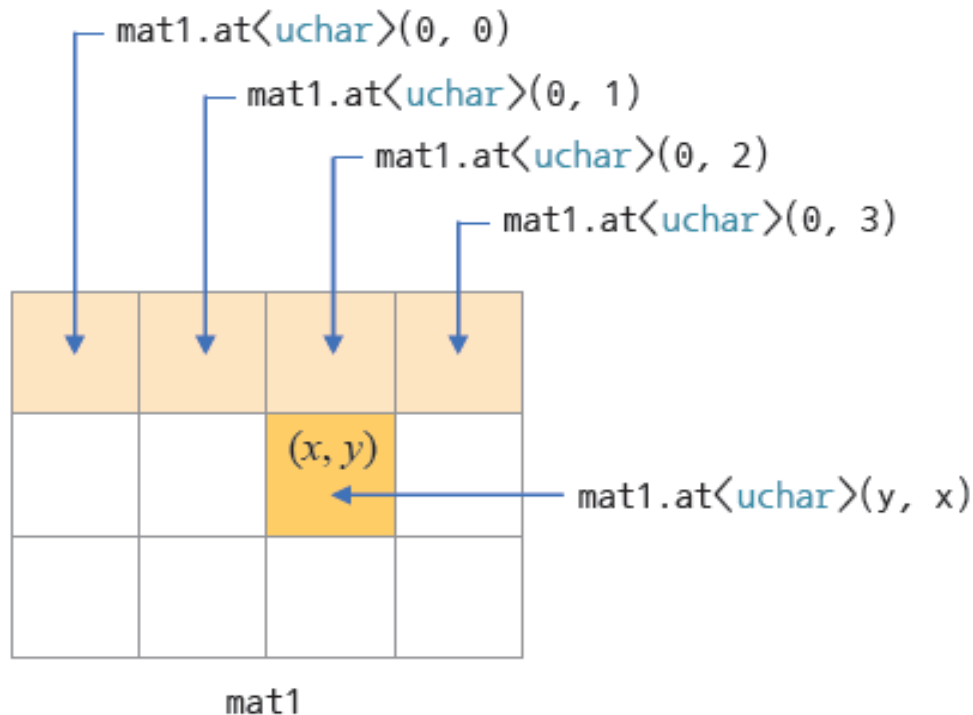


그림 3-6 Mat::at() 함수의 동작 방식

행렬의 원소 값 참조 : Mat::at() 함수이용

- 행렬 원소의 값을 화면에 출력

```
Mat src(3, 4, CV_8UC1, Scalar(0));
uchar data = src.at<uchar>(1, 2);
cout << (int)data << endl;    // 0 출력
```

- 행렬 원소에 값을 대입하는 방법 : 반환형이 참조형이므로 리턴값이 좌변에 올 수 있음

```
Mat src(3, 4, CV_8UC1, Scalar(0));
src.at<uchar>(1, 2) = 255;
cout << (int)src.at<uchar>(1, 2) << endl;    // 255 출력
```

0	0	0	0
0	0	255	0
0	0	0	0

행렬의 원소 값 참조 : Mat::at() 함수이용

- 행렬의 모든 원소의 값을 1증가시키는 예제 : Mat::at() 함수가 행렬 원소를 참조로 반환하기 때문에 Mat::at() 함수의 반환값을 변경하면 mat1 행렬 원소 값도 함께 변경

```
Mat mat1(3, 4, CV_8UC1, Scalar(0));
```

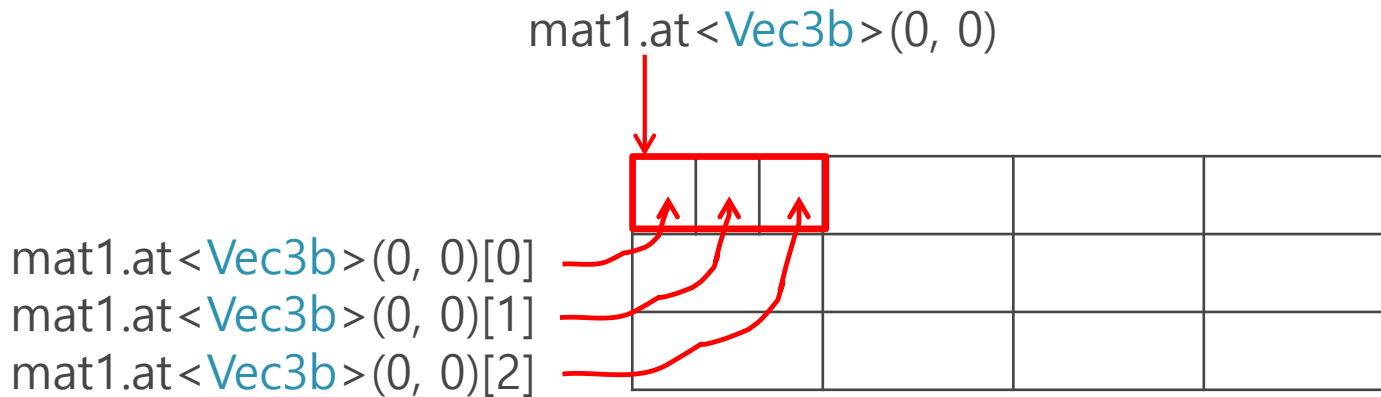
```
for (int j = 0; j < mat1.rows; j++) {
    for (int i = 0; i < mat1.cols; i++) {
        mat1.at<uchar>(j, i)++;
    }
}
```

1	1	1	1
1	1	1	1
1	1	1	1

행렬의 원소 값 참조 : Mat::at() 함수이용

- Mat 객체에 저장된 행렬의 type이 `CV_8UC3`인 경우
 - mat1 객체에서 첫 번째 원소는 `mat1.at<Vec3b>(0, 0)` 형태로 표현함

```
Mat mat1(3, 4, CV_8UC3, Scalar(0,0,0));
```



```
Vec3b p = mat1.at<Vec3b>(0, 0); ->p[0], p[1], p[2]
```

행렬의 원소 값 참조 : Mat::at() 함수이용

- Mat 객체에 그레이스케일 영상이 저장되어 있을 경우, 행렬의 원소의 타입이 **CV_8UC1** 이므로 **uchar** 자료형을 지정해야 함

```
Mat src = imread("lenna.bmp", IMREAD_GRAYSCALE);
uchar pixel = src.at<uchar>(10, 20);    //좌표(20,10)의 픽셀값
cout << (int)pixel << endl;
```

- Mat 객체에 3채널 컬러영상이 저장되어 있을 경우 행렬의 원소의 타입이 **CV_8UC3** 이므로 **Vec3b** 자료형을 지정해야 함

```
Mat src = imread("lenna.bmp", IMREAD_COLOR);
Vec3b pixel = src.at<Vec3b>(10, 20);    //좌표(20,10)의 픽셀값
cout << (int)pixel[0] << endl;          //B성분
cout << (int)pixel[1] << endl;          //G성분
cout << (int)pixel[2] << endl;          //R성분
```

행렬의 원소 값 참조 : Mat::at() 함수이용

- 그레이스케일 영상의 모든 픽셀값을 흰색으로 수정 -> 리턴값의 자료형이 참조형이므로 리턴값에 값을 대입가능

```
Mat src = imread("lenna.bmp", IMREAD_GRAYSCALE);
for (int j = 0; j < src.rows; j++)
    for (int i = 0; i < src.cols; i++)
        src.at<uchar>(j, i) = 255;
```

- 3채널 컬러영상의 모든 픽셀값을 흰색으로 수정 -> 리턴값의 자료형이 참조형이므로 리턴값에 값을 대입가능

```
Mat src = imread("lenna.bmp", IMREAD_COLOR);
for (int j = 0; j < src.rows; j++)
    for (int i = 0; i < src.cols; i++)
    {
        src.at<Vec3b>(j, i)[0] = 255;
        src.at<Vec3b>(j, i)[1] = 255;
        src.at<Vec3b>(j, i)[2] = 255;
    }
```

행렬의 원소 값 참조 : 기타방법

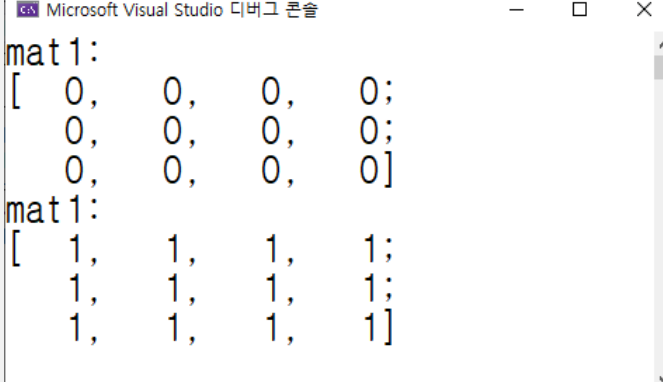
- Mat::ptr() 함수 사용
- MatIterator_ 반복자 사용
- 자세한 내용은 교재 참고

코드3-10 : 행렬 원소 참조

```
#include <opencv2/opencv.hpp>
#include <iostream>
using namespace cv;
using namespace std;
int main()
{
    Mat mat1(3, 4, CV_8UC1, Scalar(0));
    cout << "mat1:\n" << mat1 << endl;

    for (int j = 0; j < mat1.rows; j++)
        for (int i = 0; i < mat1.cols; i++)
            mat1.at<uchar>(j, i)++;

    cout << "mat1:\n" << mat1 << endl;
    return 0;
}
```



```
Microsoft Visual Studio 디버그 콘솔

mat1:
[ 0, 0, 0, 0;
  0, 0, 0, 0;
  0, 0, 0, 0]
mat1:
[ 1, 1, 1, 1;
  1, 1, 1, 1;
  1, 1, 1, 1]
```

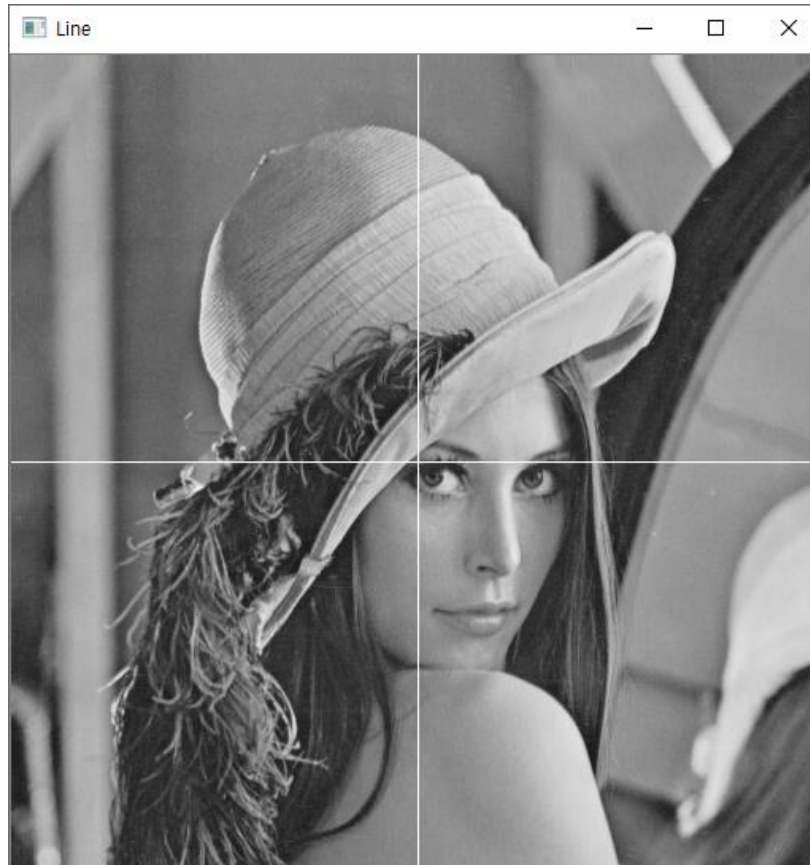
예제 : 그레이영상에 선그리기

```
#include <opencv2/opencv.hpp>
using namespace cv;
int main()
{
    Mat img = imread("lenna.bmp", IMREAD_GRAYSCALE);
    for (int i = 0; i < img.rows; i++)
        img.at<uchar>(i, img.cols/2) = 255;           //세로선 그림

    for (int i = 0; i < img.cols; i++)
        img.at<uchar>(img.rows/2, i) = 255;           //가로선 그림

    imshow("Line", img);
    waitKey(0);
    return 0;
}
```

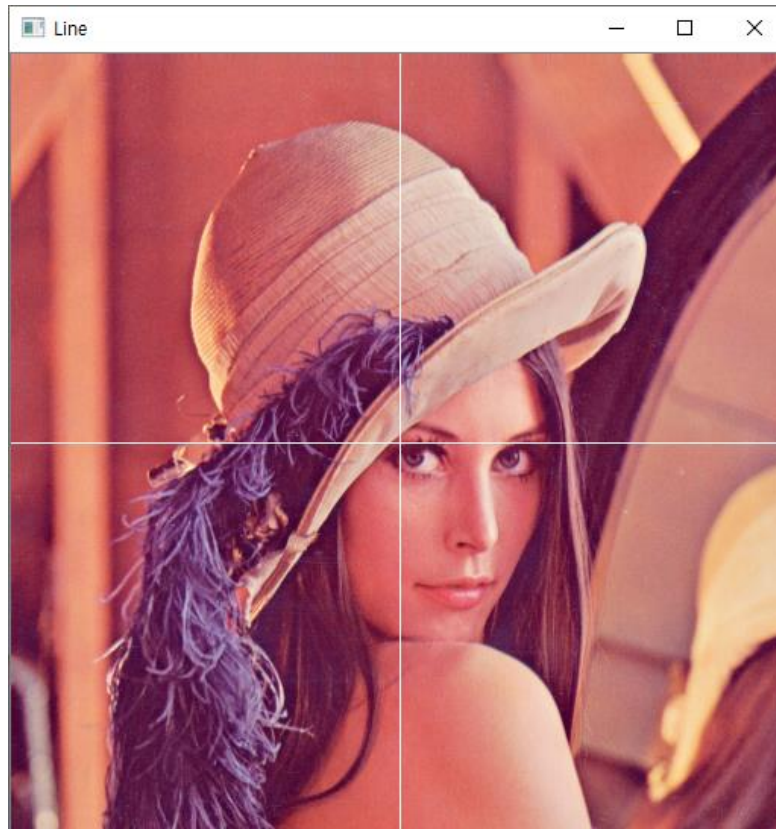
예제 : 그레이영상에 선그리기



예제 : 컬러영상에 선그리기

```
#include <opencv2/opencv.hpp>
using namespace cv;
int main()
{
    Mat img = imread("lenna.bmp", IMREAD_COLOR);
    for (int i = 0; i < img.rows; i++){
        img.at<Vec3b>(i, img.cols/2)[0] = 255;
        img.at<Vec3b>(i, img.cols/2)[1] = 255;
        img.at<Vec3b>(i, img.cols/2)[2] = 255;
    }
    for (int i = 0; i < img.cols; i++) {
        img.at<Vec3b>(img.rows/2, i)[0] = 255;
        img.at<Vec3b>(img.rows/2, i)[1] = 255;
        img.at<Vec3b>(img.rows/2, i)[2] = 255;
    }
    imshow("Line", img);
    waitKey();
    return 0;
}
```

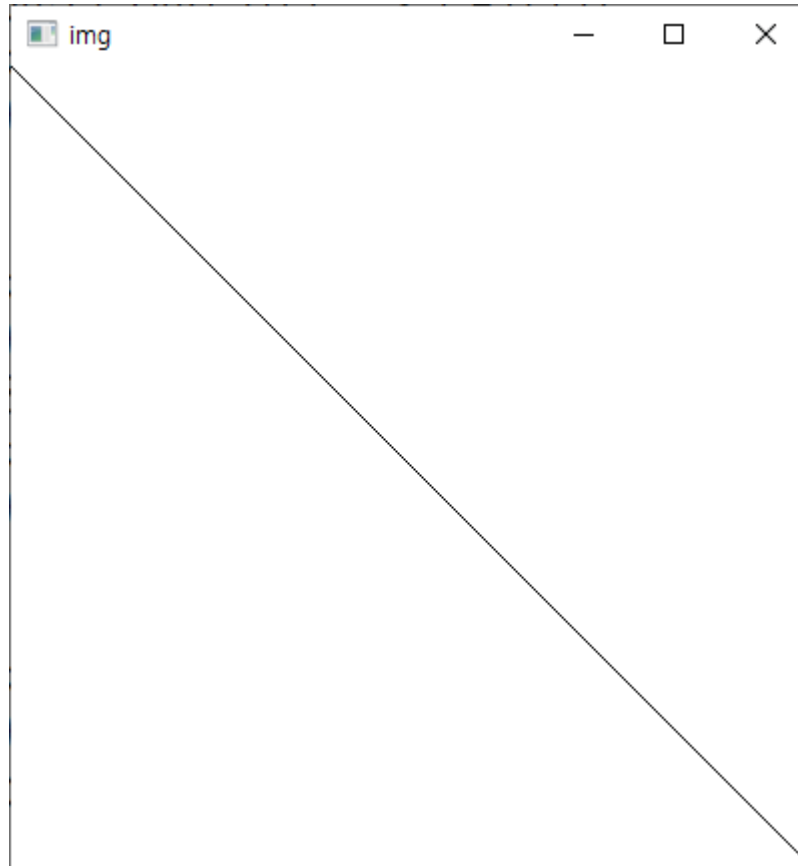

예제 : 컬러영상에 선그리기



예제 : 그래프 그리기 $y=x$

```
#include <opencv2/opencv.hpp>
#include <iostream>
using namespace cv;
using namespace std;
int main()
{
    Mat img(400, 400, CV_8UC1, Scalar(255));
    for (int x = 0; x < img.cols; x++)
    {
        int y = x;
        img.at<uchar>(y, x) = 0;
    }
    imshow("img", img);
    waitKey();
    return 0;
}
```

예제 : 그래프 그리기 $y=x$



실습과제 1

- lenna.bmp 영상에서 좌표 (50,50), (100,100), (150,150)에서 픽셀값을 아래처럼 화면에 출력하시오.
- 좌표 (x, y) -> y행 x행 원소

좌표 (50, 50)의 화소값(B,G,R) : (116, 200, 231)
좌표 (100,100)의 화소값(B,G,R) : (106, 250, 131)
좌표 (150,150)의 화소값(B,G,R) : (216, 120, 111)

실습과제2

- Lenna 영상을 그레이스케일 영상으로 변환 후 화면에 출력하고 다음 실행결과처럼 화면을 4등분하는 가로, 세로선을 그리시오. 반드시 원소 참조함수 `Mat::at()`를 사용할 것.
- 힌트 : 4등분선 위치의 픽셀값을 255로 설정하면 흰색이 된다.



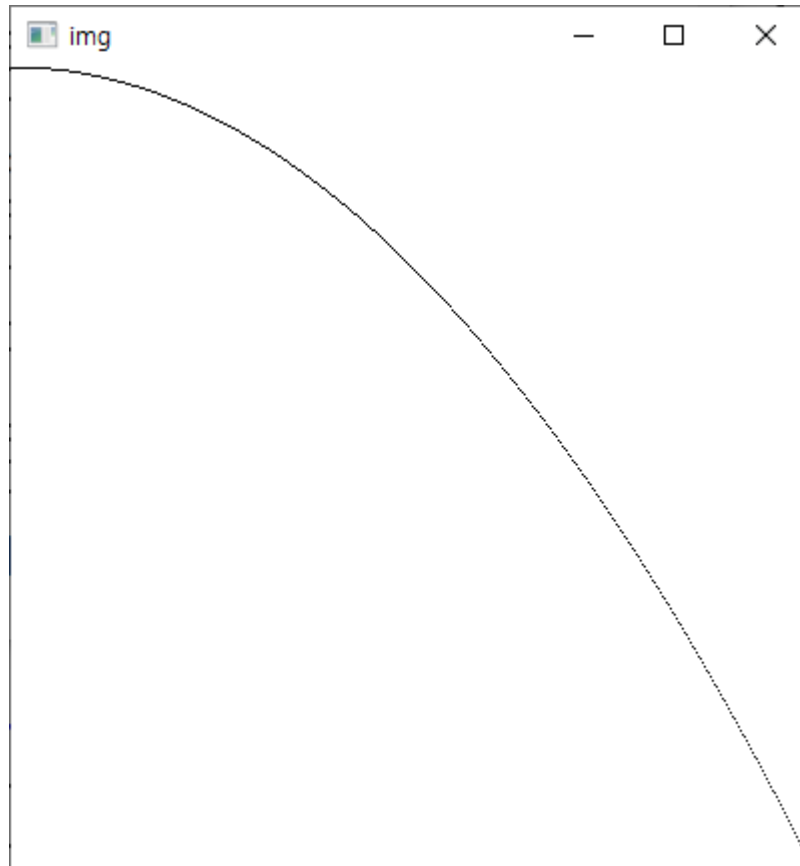
실습과제3

- Lenna 영상을 컬러 영상으로 읽은 후 화면에 출력하고 다음 실행결과처럼 화면을 4등분하는 가로, 세로선을 그리시오. 반드시 원소 참조함수 `Mat::at()`를 사용할 것.
- 힌트 : 4등분선 위치의 픽셀값을 (255, 255, 255)으로 설정하면 흰색이 된다.



실습과제4(선택문제)

- 배경색이 흰색인 400X400 영상을 만들고 $y=1/400*x^2$ 의 그래프를 그리시오.



과제 제출 방법

- 소스파일, 라인단위의 코드설명, 실행결과를 하나의 PDF 파일로 작성하여 과제게시판에 제출하시오. 이외 양식은 0점 처리함
- 제출기한은 과제 게시판을 참고할 것.